AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm drzewa binarnego (BST) z zastosowaniem GitHub

Autor:
Jakub Podkówka, Jakub Pacułt

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Wprowadzenie teoretyczne i cel projektu 3
1.2. Szczegółowe wymagania funkcjonalne 3
1.3. Architektura modularna i struktura klas 4
1.4. Algorytmy przechodzenia oraz operacji na plikach 6
1.5. Wymagania dotyczące pracy w zespole i systemu Git 8
1.6. Dokumentacja kodu za pomocą systemu Doxygen 8

2. Analiza problemu **10**

2.1. Zastosowanie drzew poszukiwań binarnych w informatyce 10
2.2. Sposób działania algorytmu i podstawowe operacje 11
2.3. Przykładowy przebieg działania algorytmu krok po kroku 12
2.4. Opis narzędzi: System kontroli wersji Git oraz platforma GitHub 14

3. Projektowanie **17**

3.1. Środowisko programistyczne i konfiguracja kompilatora 17
3.2. Diagram klas UML i architektura systemu 17
3.3. Przepływ pracy i sposób użycia narzędzia Git 17

4. Implementacja **19**

5. Wnioski **20**

Literatura **21**

Spis rysunków **21**

Spis tabel **22**

Spis listingów **23**

1. Ogólne określenie wymagań

Niniejszy rozdział przedstawia szczegółowe opisy wymagań technicznych, funkcjonalnych. Głównym zadaniem jest stworzenie w pełni funkcjonalnej aplikacji w języku C++ operującej na strukturze danych drzewa poszukiwań binarnych (ang. *Binary Search Tree* – BST) z wykorzystaniem kontenerów biblioteki standardowej STL, a także przeprowadzenie pełnego cyklu integracji i kontroli wersji za pomocą platformy GitHub w zespole dwuosobowym.

1.1. Wprowadzenie teoretyczne i cel projektu

Drzewo poszukiwań binarnych jest dynamiczną strukturą danych, w której każdy węzeł zawiera klucz (w naszym przypadku wartość liczbową) oraz co najwyżej dwóch potomków, nazywanych tradycyjnie lewym i prawym dzieckiem. Kluczową własnością drzewa BST jest porządek, w którym dla każdego węzła N:

- Wszystkie klucze znajdujące się w lewym poddrzewie węzła N mają wartości ściśle mniejsze od wartości klucza węzła N.
- Wszystkie klucze znajdujące się w prawym poddrzewie węzła N mają wartości większe lub równe wartości klucza węzła N.

Właściwość ta pozwala na wysoce wydajne wyszukiwanie elementów, osiągające złożoność obliczeniową $O(\log_2 n)$ w przypadku drzew zrównoważonych. W najgorszym przypadku (gdy wprowadzane dane są już posortowane, a drzewo ulega zdegenerowaniu do postaci listy jednokierunkowej) złożoność ta wzrasta do $O(n)$. Cel projektu obejmuje nie tylko implementację samego drzewa, ale również opracowanie stabilnego modułu serializacji danych (zapis/odczyt binarny oraz tekstowy) oraz stworzenie przejrzystego interfejsu konsolowego z obsługą menu.

1.2. Szczegółowe wymagania funkcjonalne

Wymagania postawione przed aplikacją podzielono na grupy funkcjonalne, które zostały wyszczególnione w poniższym zestawieniu:

1. **Zarządzanie strukturą drzewa BST:**
    - Dynamiczne dodawanie nowego elementu (węzła) do istniejącego lub pustego drzewa z zachowaniem własności BST.

- Usuwanie wskazanego elementu z drzewa z poprawną reorganizacją wskaźników (przypadki: węzeł bez dzieci, z jednym dzieckiem, z dwójką dzieci).
- Całkowite czyszczenie drzewa (zwolnienie pamięci wszystkich węzłów, np. przy użyciu przejścia *postorder*).
- Wyszukiwanie ścieżki od korzenia do wybranego elementu i prezentacja kolejnych węzłów na tej ścieżce.

2. **Wizualizacja i reprezentacja danych:**

   - Wyświetlanie graficzne lub strukturalne drzewa w konsoli.
   - Wybór metody przechodzenia przez użytkownika podczas wyświetlania: *preorder* (root-left-right), *inorder* (left-root-right) oraz *postorder* (left-right-root).

3. **Obsługa plików i wejścia/wyjścia (I/O):**

   - Zapis wygenerowanego drzewa do czytelnego pliku tekstowego (`.txt`).
   - Dedykowana klasa narzędziowa do binarnego zapisu i odczytu drzewa (`.bin`).
   - Wczytywanie pliku tekstowego z cyframi/liczbami w celu zbudowania nowego drzewa lub dołączenia danych do drzewa istniejącego.

4. **Interfejs i architektura programu:**

   - Funkcja `main` serwująca interaktywne menu tekstowe z pętlą obsługi zdarzeń i bezpiecznym wyjściem z programu.
   - Pełny podział modularny: każda klasa zdeklarowana w odrębnym pliku nagłówkowym (`.h`) i zaimplementowana w pliku źródłowym (`.cpp`).

1.3. Architektura modularna i struktura klas

Zgodnie z zasadami programowania obiektowego (OOP) aplikacja została podzielona na trzy główne komponenty. Taki podział gwarantuje wysoką spójność wewnętrzną (ang. *cohesion*) oraz niskie sprzężenie (ang. *coupling*).

Wykaz poszczególnych modułów programu przedstawiono w tabeli 1.1 (s. 5).

Listing 1 (s. 5) prezentuje przykładowy interfejs klasy `BSTTree` zawierający deklaracje kluczowych metod operujących na drzewie.

Tab. 1.1. Struktura klas i modułów aplikacji

Nazwa pliku	Moduł / Klasa	Odpowiedzialność w systemie
`BST.h / .cpp`	`BSTTree`	Klasa główna drzewa: wstawianie, usuwanie, przechodzenie, wyszukiwanie ścieżki.
`FileManager.h / .cpp`	`FileHandler`	Zapis/odczyt binarny reprezentacji drzewa, wczytywanie liczb z pliku tekstowego.
`Node.h`	Node (struktura)	Przechowuje dane oraz położenie elementu
`main.cpp`	Interfejs (CLI)	Główna pętla programu, sterowanie menu, przyjmowanie danych od użytkownika.

```
#ifndef BST_H
#define BST_H

#include <iostream>
#include <vector>
#include <string>

struct Node {
    int value;
    Node* left;
    Node* right;
    Node(int val) : value(val), left(nullptr), right(nullptr) {}
};

class BSTTree {
private:
    Node* root;
    void clearRecursive(Node* node);
    void preorderRecursive(Node* node) const;
    void inorderRecursive(Node* node) const;
    void postorderRecursive(Node* node) const;
    bool findPathRecursive(Node* node, int target, std::vector<int
   >& path) const;

public:
    BSTTree();
```

```
26      ~BSTTree();
27
28      void insert(int value);
29      bool remove(int value);
30      void clear();
31      std::vector<int> getPath(int target) const;
32
33      void displayPreorder() const;
34      void displayInorder() const;
35      void displayPostorder() const;
36
37      Node* getRoot() const { return root; }
38  };
39
40  #endif // BST_H
```

Listing 1. Deklaracja nagłówkowa klasy BSTTree

1.4. Algorytmy przechodzenia oraz operacji na plikach

Kluczowym elementem projektu jest implementacja trzech podstawowych algorytmów przejścia przez strukturę drzewiastą. Każda z tych metod realizuje specyficzny porządek przetwarzania danych:

- **Preorder (współrzędny):** Przetwarzany jest najpierw węzeł bieżący, następnie jego lewe poddrzewo, a na końcu prawe poddrzewo. Przejście to jest wykorzystywane m.in. do tworzenia kopii drzewa lub do zapisu w pliku w sposób umożliwiający odtworzenie identycznej struktury.
- **Inorder (poprzeczny):** Najpierw przetwarzane jest lewe poddrzewo, potem węzeł bieżący, a na końcu prawe poddrzewo. Dla drzewa BST przejście to gwarantuje wypisanie wszystkich zgromadzonych wartości w kolejności rosnącej.
- **Postorder (odwrotny):** Najpierw przetwarzane są lewe i prawe poddrzewo, a dopiero na końcu węzeł bieżący. Jest to idealne rozwiązanie przy zwalnianiu pamięci (usuwaniu drzewa), gdyż dzieci są niszczone przed swoim rodzicem.

W drugiej klasie (`FileHandler`) zaimplementowano funkcjonalność operowania na plikach w dwóch standardach. Tabela 1.2 (s. 7) zawiera opis zastosowania poszczególnych formatów zapisu.

Listing 2 (s. 7) przedstawia fragment kodu odpowiedzialny za binarny zapis struktury drzewa do pliku.

Tab. 1.2. Porównanie formatów plików wykorzystywanych w projekcie

Cecha	Plik Tekstowy (.txt)	Plik Binarny (.bin)
Forma danych	Znaki ASCII (cyfry oddzielone spacją/nową linią)	Bajtowa reprezentacja struktur w pamięci
Zastosowanie	Wczytywanie danych wejściowych, eksport widoku	Szybka serializacja i deserializacja stanu drzewa
Odczytywalność	Bezpośrednio widoczna w edytorze tekstu	Wymaga dedykowanego parsera / programu

```cpp
#include "FileHandler.h"
#include <fstream>

bool FileHandler::saveBinary(const std::string& filename, const BSTTree& tree) {
    std::ofstream outFile(filename, std::ios::binary);
    if (!outFile.is_open()) {
        return false;
    }

    // Zapytanie o strukture i zapis pre-order do pliku binarnego
    std::vector<int> data;
    // ... zbieranie danych do wektora ...

    size_t count = data.size();
    outFile.write(reinterpret_cast<const char*>(&count), sizeof(count));
    for (int val : data) {
        outFile.write(reinterpret_cast<const char*>(&val), sizeof(val));
    }

    outFile.close();
    return true;
}
```

Listing 2. Zapis binarny węzłów drzewa BST do pliku

1.5. Wymagania dotyczące pracy w zespole i systemu Git

Głównym celem dydaktycznym w zakresie inżynierii oprogramowania jest zdobycie praktycznych umiejętności pracy z systemem kontroli wersji Git oraz platformą GitHub. Wytyczne projektu narzucają rygorystyczny przebieg prac programistycznych:

1. **Minimalna liczba commitów:** Każdy z dwóch członków zespołu zobowiązany jest do wykonania co najmniej 5 przejrzystych i opatrzonych opisem commitów na swoich gałęziach roboczych.

2. **Praca sekwencyjna na gałęziach:**

   - Osoba A tworzy własną gałąź roboczą, wykonuje na niej kilka commitów, a następnie scala ją do głównej gałęzi (`main`).
   - Po ukończeniu scalenia przez Osobę A, Osoba B tworzy nową gałąź roboczą, realizuje zaplanowane zadania i wykonuje scalenie do gałęzi głównej.

3. **Praca równoległa:** Obie osoby w zespole jednocześnie tworzą nowe gałęzie robocze (rozpoczynając pracę w tym samym punkcie historii projektu) i prowadzą niezależne prace programistyczne.

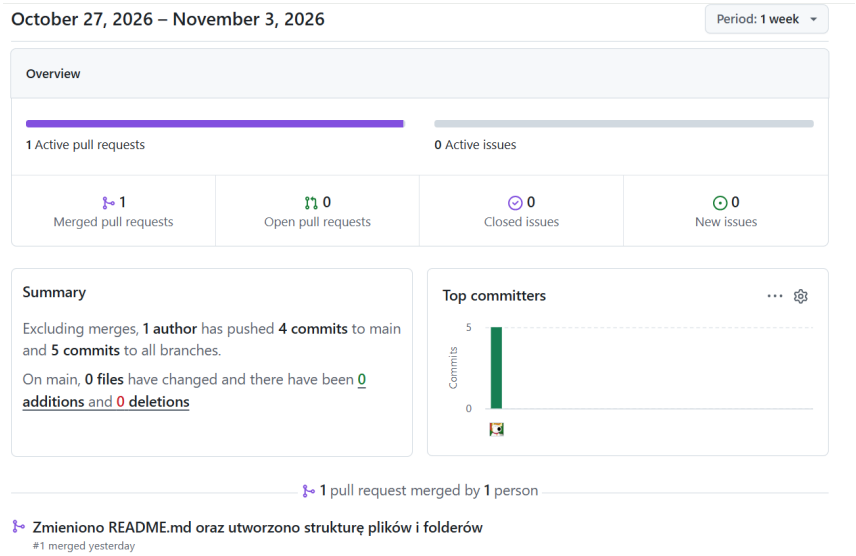
4. **Generowanie i rozwiązywanie konfliktów:**

   - W trakcie pracy równoległej celowo modyfikowane są te same linie plików źródłowych, co wymusza powstanie konfliktów przy scalaniu (ang. *merge conflict*).
   - Wymagane jest wygenerowanie i poprawne rozwiązanie łącznie co najmniej 6 konfliktów (dokładnie 3 przez Osobę A oraz 3 przez Osobę B).
   - Rozwiązywanie konfliktów przeprowadzane jest z użyciem wtyczki GitHub w środowisku Visual Studio Code lub Visual Studio.

Na rysunku 1.1 (s. 9) przedstawiono przykładowy schemat drzewa commitów z repozytorium GitHub po scaleniu gałęzi prac równoległych.

1.6. Dokumentacja kodu za pomocą systemu Doxygen

Do projektu należy dołączyć automatycznie wygenerowaną dokumentację techniczną. Wykorzystywany jest do tego system **Doxygen**, który agreguje specjalne bloki komentarzy umieszczone w plikach nagłówkowych `.h` i źródłowych `.cpp`.



Rys. 1.1. Wizualizacja historii commitów i scalania gałęzi z platformy GitHub

Tab. 1.3. Najważniejsze znaczniki systemu Doxygen stosowane w dokumencie

Znacznik Doxygen	**Opis i funkcja w dokumencie**
`@file`	Oznacza plik źródłowy lub nagłówkowy podlegający indeksowaniu.
`@class`	Opisuje przeznaczenie i zakres odpowiedzialności danej klasy.
`@brief`	Krótki, jednozdaniowy opis metody lub struktury.
`@param`	Opisuje parametr wejściowy przekazywany do metody.
`@return`	Opisuje wartość zwracaną przez funkcję lub metodę.

Wykaz stosowanych tagów Doxygen przedstawia tabela 1.3 (s. 9).

Podsumowując, zrealizowany w powyższy sposób rozdział wprowadza w kompletny zakres wymagań projektowych. Wszystkie operacje na drzewie BST, obsługa plików binarnych oraz procedury gitowe zostaną w kolejnych rozdziałach poddane szczegółowej analizie implementacyjnej i testowej.

2. Analiza problemu

W niniejszym rozdziale dokonano szczegółowej analizy teoretycznej oraz praktycznej struktury danych w postaci drzewa poszukiwań binarnych (ang. *Binary Search Tree* – BST). Omówiono praktyczne obszary zastosowań tego algorytmu w nowoczesnej informatyce, formalny sposób działania poszczególnych operacji, jak również przedstawiono pełny przykład wykonania operacji na drzewie krok po kroku. Ponadto opisano narzędzia inżynierii oprogramowania wykorzystane w trakcie realizacji projektu, w tym system kontroli wersji Git, platformę GitHub oraz generator dokumentacji Doxygen.

2.1. Zastosowanie drzew poszukiwań binarnych w informatyce

Drzewo poszukiwań binarnych jest jedną z najbardziej fundamentalnych dynamicznych struktur danych. Ze względu na swoje właściwości wyszukiwania oraz możliwości szybkiej modyfikacji, algorytmy oparte na drzewach BST znajdują szerokie zastosowanie w wielu gałęziach informatyki:

1. **Systemy zarządzania bazami danych (DBMS) i indeksowanie:** Struktury wariantowe drzew BST (takie jak drzewa AVL, drzewa czerwono-czarne czy B-drzewa) stanowią podstawę indeksowania relacyjnych baz danych. Pozwalają na natychmiastowe zlokalizowanie rekordu o określonym kluczu w czasie logarytmicznym $O(\log n)$ bez konieczności liniowego przeglądania całej bazy.

2. **Implementacja słowników i zbiorów w bibliotekach standardowych:** Wiele języków programowania wykorzystuje zrównoważone drzewa BST do realizacji kontenerów asocjacyjnych. Przykładem jest biblioteka STL w języku C++, gdzie kontenery takie jak `std::map` oraz `std::set` są domyślnie zaimplementowane w oparciu o drzewa czerwono-czarne.

3. **Kompresja danych (Drzewa Huffmana):** W algorytmach bezstratnej kompresji danych, takich jak kodowanie Huffmana, dynamiczne struktury drzewiaste są wykorzystywane do budowania prefiksowych kodów znaków o zmiennej długości na podstawie częstotliwości ich występowania.

4. **Drzewa wyrażeń i parsowanie składniowe (AST):** W kompilatorach i interpreterach języków programowania drzewa są wykorzystywane jako abs-

trakcyjne drzewa składniowe (ang. *Abstract Syntax Tree* – AST) do reprezentowania struktury gramatycznej kodu źródłowego oraz ewaluacji wyrażeń arytmetycznych i logicznych.

5. **Grafika komputerowa i podział przestrzeni (BSP):** Drzewa binarnego podziału przestrzeni (ang. *Binary Space Partitioning* – BSP) pomagają w sortowaniu i wyświetlaniu obiektów trójwymiarowych w kolejności od przodu do tyłu lub odwrotnie, eliminując potrzebę wykonywania kosztownych obliczeń widoczności powierzchni.

2.2. Sposób działania algorytmu i podstawowe operacje

Struktura BST składa się z węzłów połączonych wskaźnikami. Każdy węzeł zawiera przechowywaną wartość (`data`) oraz dwa wskaźniki poddrzew: lewego (`left`) i prawego (`right`). Nadrzędną zasadą porządkującą strukturę jest relacja:

$$\forall_{v \in \mathrm{Left}(N)} \mathrm{val}(v) < \mathrm{val}(N) \quad \text{oraz} \quad \forall_{u \in \mathrm{Right}(N)} \mathrm{val}(u) \geq \mathrm{val}(N) \tag{2.1}$$

Poniżej opisano mechanizm działania kluczowych algorytmów zaimplementowanych w klasie `BSTTree`:

- **Dodawanie elementu (Insertion):** Rozpoczyna się od korzenia. Jeśli drzewo jest puste, nowy węzeł staje się korzeniem. W przeciwnym razie porównuje się wstawianą wartość z wartością w aktualnym węźle. Jeśli jest mniejsza, przechodzi się rekurencyjnie do lewego poddrzewa; jeśli jest większa lub równa – do prawego. Proces powtarza się do momentu natrafienia na wskaźnik `nullptr`, w miejscu którego tworzony jest nowy węzeł.
- **Wyszukiwanie drogi do elementu (Search / Path finding):** Algorytm rozpoczyna przeszukiwanie od korzenia, dopisując każdy odwiedzany węzeł do wektora ścieżki. Przechodzenie w dół drzewa odbywa się zgodnie z relacją porządku. Wyszukiwanie kończy się powodzeniem po odnalezieniu szukanego klucza lub niepowodzeniem po osiągnięciu końca gałęzi (`nullptr`).
- **Algorytmy przechodzenia drzewa (Traversals):**
  - *Preorder (korzeń, lewe, prawe):* Służy do kopiowania lub zapisu struktury drzewa.
  - *Inorder (lewe, korzeń, prawe):* Zwraca uporządkowaną (rosnącą) sekwencję elementów.

  - *Postorder (lewe, prawe, korzeń):* Wykorzystywany przy bezpiecznym zwalnianiu pamięci węzłów.

- **Usuwanie elementu (Deletion):** Jest to najbardziej złożona operacja, wymagająca rozpatrzenia trzech odrębnych przypadków:

  1. *Usuwanie liścia (węzeł bez potomków):* Wystarczy usunąć węzeł z pamięci i ustawić wskaźnik rodzica na `nullptr`.
  2. *Usuwanie węzła z jednym dzieckiem:* Wskaźnik z rodzica usuwanego węzła zostaje przekierowany bezpośrednio na jego jedyne dziecko, po czym usuwany węzeł jest zwalniany.
  3. *Usuwanie węzła z dwojgiem dzieci:* Należy odnaleźć bezpośrednika usuwanego węzła – zazwyczaj jest to najmniejszy element w prawym poddrzewie (tzw. następnik, ang. *inorder successor*). Wartość następnika jest kopiowana do usuwanego węzła, a następnie rekurencyjnie usuwa się następnik (który ma co najwyżej jedno dziecko).

2.3. Przykładowy przebieg działania algorytmu krok po kroku

W celu zaprezentowania poprawnego działania zaimplementowanych algorytmów prześledzono proces tworzenia drzewa BST oraz wykonania na nim serii operacji dla konkretnego zestawu danych wejściowych.

Zestaw danych wejściowych stanowi ciąg 7 liczb całkowitych przekazany do pustego drzewa w następującej kolejności:

$$S = \{50, 30, 70, 20, 40, 60, 80\} \tag{2.2}$$

Przebieg budowy drzewa BST krok po kroku przedstawia tabela 2.1 (s. 13).

Po wstawieniu wszystkich elementów otrzymujemy drzewo o wysokości $h = 2$, którego stan tekstowy w poszczególnych metodach przechodzenia prezentuje się następująco:

- **Preorder:** $50 \to 30 \to 20 \to 40 \to 70 \to 60 \to 80$
- **Inorder:** $20 \to 30 \to 40 \to 50 \to 60 \to 70 \to 80$ (uporządkowane)
- **Postorder:** $20 \to 40 \to 30 \to 60 \to 80 \to 70 \to 50$

Kolejnym krokiem w przykładzie jest **wyszukanie drogi do elementu 40**:

1. Start w korzeniu (wartość 50). Dodanie 50 do ścieżki.

Tab. 2.1. Analiza budowy drzewa BST krok po kroku

Krok	**Wartość**	**Stan struktury i podjęte decyzje algorytmiczne**
1	50	Drzewo puste. Utworzenie korzenia z wartością 50.
2	30	$30 < 50 \rightarrow$ wstawienie jako lewe dziecko węzła 50.
3	70	$70 > 50 \rightarrow$ wstawienie jako prawe dziecko węzła 50.
4	20	$20 < 50 \rightarrow$ lewo; $20 < 30 \rightarrow$ lewe dziecko węzła 30.
5	40	$40 < 50 \rightarrow$ lewo; $40 > 30 \rightarrow$ prawe dziecko węzła 30.
6	60	$60 > 50 \rightarrow$ prawo; $60 < 70 \rightarrow$ lewe dziecko węzła 70.
7	80	$80 > 50 \rightarrow$ prawo; $80 > 70 \rightarrow$ prawe dziecko węzła 70.

2. Szukana wartość $40 < 50$, przechodzimy w lewo do węzła 30. Dodanie 30 do ścieżki.

3. Szukana wartość $40 > 30$, przechodzimy w prawo do węzła 40. Dodanie 40 do ścieżki.

4. Odnaleziono szukaną wartość $40 == 40$.

5. Zwrócona ścieżka: `[50, 30, 40]`.

Następnie wykonujemy operację **usunięcia węzła 30** (węzeł posiadający dwoje dzieci: 20 oraz 40):

1. Zlokalizowanie węzła 30 w lewym poddrzewie korzenia.

2. Odnalezienie następnika w prawym poddrzewie węzła 30. Najmniejsza wartość w prawym poddrzewie węzła 30 to węzeł 40.

3. Skopiowanie wartości 40 do węzła 30.

4. Usunięcie oryginalnego węzła 40 (który był liściem).

Stan drzewa po usunięciu elementu 30 w przejściu *Inorder* wynosi:

$$\text{Inorder}_{\text{po usunięciu}} = \{20, 40, 50, 60, 70, 80\} \tag{2.3}$$

Zimplementowaną logikę usuwania węzła z dwójką dzieci przedstawia poniższy fragment kodu w języku C++ (listing 3).

```
Node* BSTTree::removeNode(Node* node, int key) {
    if (node == nullptr) return nullptr;

    if (key < node->value) {
        node->left = removeNode(node->left, key);
    } else if (key > node->value) {
        node->right = removeNode(node->right, key);
    } else {
        // Wezel znaleziony
        if (node->left == nullptr) {
            Node* temp = node->right;
            delete node;
            return temp;
        } else if (node->right == nullptr) {
            Node* temp = node->left;
            delete node;
            return temp;
        }
        // Case: Dwoje dzieci - szukanie nastepnika (minValueNode)
        Node* temp = findMin(node->right);
        node->value = temp->value;
        node->right = removeNode(node->right, temp->value);
    }
    return node;
}
```

Listing 3. Algorytm usuwania węzła z dwójką dzieci w klasie BSTTree

2.4. Opis narzędzi: System kontroli wersji Git oraz platforma GitHub

Współczesna inżynieria oprogramowania wymaga stosowania zaawansowanych narzędzi wspomagających pracę grupową, automatyzację testów oraz kontrolę jakości kodu. W realizowanym projekcie wykorzystano następujący zestaw narzędzi programistycznych:

1. **Git (Rozproszony System Kontroli Wersji):** Git śledzi historię zmian

w plikach projektowych. Każda zmiana jest rejestrowana w postaci bezpiecznego commita opatrzonego unikalnym sumarycznym kodem skrótu SHA-1. Kluczowe pojęcia architektoniczne systemu Git obejmują:

- *Working Directory (Katalog roboczy):* Pliki na dysku, na których programista bezpośrednio wykonuje pracę.
- *Staging Area / Index (Poczekalnia):* Obszar przygotowawczy, do którego za pomocą polecenia `git add` trafiają pliki przeznaczone do najnowszego commita.
- *Local Repository (Lokalne repozytorium):* Baza danych na lokalnym dysku przechowująca pełną historię wersji po wykonaniu polecenia `git commit`.
- *Remote Repository (Zdalne repozytorium):* Centralna kopia projektu na serwerze (GitHub).

2. **Platforma GitHub i praca na gałęziach (Branching):** GitHub umożliwia udostępnianie kodu, prowadzenie dyskusji oraz integrację prac programistów. Izolacja wprowadzanych funkcji realizowana jest poprzez gałęzie (*branches*). Praca w projekcie dwuosobowym wymagała opanowania mechanizmu scalania (*git merge*) oraz radzenia sobie z konfliktami.

   Konflikt powstaje, gdy dwie osoby zmodyfikują te same linie w tym samym pliku na odrębnych gałęziach. Wtedy system Git zatrzymuje proces automatycznego scalania i nakłada znaczniki konfliktu:

```
<<<<<<< HEAD
    // Kod rozwiniety przez Osobe A na galezi glownej
=======
    // Kod zmodyfikowany przez Osobe B na galezi roboczej
>>>>>>> feature-branch
```

   Rozwiązanie konfliktu polega na ręcznej analizie zmian, wyborze właściwej wersji kodu i ponownym wykonaniu zatwierdzenia commitem.

3. **Integracja ze środowiskiem IDE (Visual Studio / VS Code):** Do zarzadzania repozytorium wykorzystano wbudowaną wtyczkę Git w środowiskach Microsoft Visual Studio / Visual Studio Code. Umożliwia ona wizualne porównywanie różnic w kodzie (ang. *diff*), szybkie rozwiązywanie konfliktów przyciskami „Accept Current Change" / „Accept Incoming Change" oraz graficzne przeglądanie drzewa historii commitów.

4. **Generator dokumentacji Doxygen:** Doxygen jest narzędziem służącym do automatycznego tworzenia dokumentacji technicznej w formatach HTML i LaTeX bezpośrednio z komentarzy umieszczonych w kodzie źródłowym C++. Użycie specjalnych tagów (np. `@brief`, `@param`, `@return`) pozwala uzyskać przejrzystą, czytelną dla innych programistów specyfikację klas.

Podsumowując analizę problemu, omówione algorytmy i zasady operowania na strukturze BST oraz poznane narzędzia kontroli wersji stanowią fundament do dalszego opisu implementacji i wyników testów w kolejnych rozdziałach pracy.

3. Projektowanie

W niniejszym rozdziale zaprezentowano etap projektowy aplikacji, obejmujący zestawienie użytych narzędzi, konfiguracyjne ustawienia kompilatora, architekturę klas wyrażoną diagramem UML oraz opis procedury zarządzania repozytorium w systemie kontroli wersji.

3.1. Środowisko programistyczne i konfiguracja kompilatora

Do realizacji projektu wykorzystano standard języka **C++17**, zapewniający wsparcie dla kontenerów STL (`std::vector`, `std::string`) oraz ulepszoną obsługę strumieni plików (`std::fstream`). Zespół pracował w środowiskach Microsoft Visual Studio 2022 oraz Visual Studio Code z wykorzystaniem kompilatorów MSVC oraz MinGW (GCC 12.2).

Konfiguracja kompilatora obejmowała następujące flagi budowania projektu:

- `-std=c++17` (lub `/std:c++17`) – wymuszenie standardu C++17.
- `-Wall -Wextra` (lub `/W4`) – włączenie szczegółowych ostrzeżeń kompilatora w celu wykrycia potencjalnych wycieków pamięci i niezainicjalizowanych wskaźników.
- `-O2` – flaga optymalizacyjna dla wersji ostatecznej (Release).

3.2. Diagram klas UML i architektura systemu

Architektura aplikacji opiera się na ścisłym podziale odpowiedzialności. Główna logika drzewa została odizolowana od modułu I/O oraz warstwy interfejsu. Rysunek 3.1 (s. 18) przedstawia diagram klas UML opisujący powiązania pomiędzy strukturami programu.

3.3. Przepływ pracy i sposób użycia narzędzia Git

Praca w zespole dwuosobowym odbywała się z wykorzystaniem modelu *Feature Branch Workflow*. Poniżej opisano sekwencję poleceń używanych podczas tworzenia nowych funkcjonalności i rozwiązywania konfliktów:

1. **Tworzenie gałęzi:** `git checkout -b feature/nazwa_funkcji` – praca na wyizolowanej gałęzi roboczej.

2. **Zatwierdzanie etapów:** `git add .` oraz `git commit -m "Opis zmian"` – tworzenie lokalnych punktów kontrolnych.

3. **Publikacja i scalanie:** `git push origin feature/...` oraz wywołanie `git merge` do gałęzi `main`.

4. **Rozwiązywanie konfliktów:** W przypadku wystąpienia konfliktu scalania używano wbudowanego edytora *Merge Editor* w środowisku VS Code/Visual Studio do ręcznego wybrania poprawnych linii kodu.

4. Implementacja

5. Wnioski

Spis rysunków

1.1. Wizualizacja historii commitów i scalania gałęzi z platformy GitHub . 9

3.1. Diagram klas UML aplikacji drzewa BST 18

Spis tabel

1.1. Struktura klas i modułów aplikacji 5
1.2. Porównanie formatów plików wykorzystywanych w projekcie 7
1.3. Najważniejsze znaczniki systemu Doxygen stosowane w dokumencie . 9
2.1. Analiza budowy drzewa BST krok po kroku 13

Spis listingów

1. Deklaracja nagłówkowa klasy BSTTree 5
2. Zapis binarny węzłów drzewa BST do pliku 7
3. Algorytm usuwania węzła z dwójką dzieci w klasie BSTTree 14